

Intelligent Personal AI Assistant



Name	ID	Intake	Section
MD Tanvir Islam	20245103244	53	7
Raisa Sadik Smrity	20245103248	53	7
Sourav Chandra Shil	20245103254	53	7
Tasmia Anwar Nisa	20245103261	53	7
Syed Sami Sayem	20245103247	53	7

SUPERVISED BY :

Rubaiya Reza Sohana

Department of CSE

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING BANGLADESH UNIVERSITY OF
BUSINESS AND TECHNOLOGY (BUBT)**

ABSTRACT

Zunayra is a personal artificial intelligence assistant designed to provide conversational interaction while maintaining selected long-term information about the user. The supplied implementation consists of two related Python applications: a standalone local assistant and a Gradio-based browser chat interface. Both versions load a locally stored causal language model through Hugging Face Transformers and PyTorch. The system can generate conversational responses, maintain conversation history, extract meaningful user facts into a structured JSON memory file, and optionally retrieve fresh information from the web through the Tavily API.

A major feature of Zunayra is its separation between the assistant's identity and the user's remembered profile. The implementation contains explicit grounding rules intended to prevent the model from confusing facts about the user with information about itself. It also supports an assistant nickname, selective memory extraction, a "forget" mechanism, rolling conversation summarization, and a decision layer that determines when current web information may be necessary.

The browser version uses Gradio to provide a ChatGPT/Gemini-style interface. User messages are displayed immediately, after which model processing, memory checks, optional web search, and response generation occur. The standalone version provides the same core capabilities through a command-line interface. Together, these components demonstrate a practical architecture for building a local personal AI assistant with persistent state while keeping the primary model execution on the local machine.

1. INTRODUCTION

Artificial intelligence has increasingly moved from specialized laboratory systems into practical software applications. Personal AI assistants are one example of this transition. A modern personal assistant is expected not only to generate natural-language responses but also to maintain conversational context, remember useful user preferences, and retrieve current information when a static model may not be sufficient.

Zunayra was developed as a personal AI assistant with these practical requirements in mind. The implementation is based on Python and uses Hugging Face Transformers to load a local causal language model. The model is loaded from a configurable local directory, allowing the assistant to operate without requiring the language model itself to be downloaded at every execution. The standalone application can be launched with a Python command, while the user-interface version opens a browser-based chat page through Gradio. filecite turn0file0 L2-L18

The project is particularly focused on three areas: conversational generation, persistent memory, and optional external information retrieval. The memory subsystem stores structured facts in JSON. The conversation subsystem keeps recent exchanges and summarizes older exchanges when the configured history limit is exceeded. The web-search subsystem uses Tavily only when a search is required and an API key is available. These components are coordinated by a main processing pipeline.

1.1 Background

A language model normally generates an answer from the input and the context supplied to it. A personal assistant, however, benefits from additional state. For example, a user may tell the assistant a preferred name, hobby, project, or favorite item. Zunayra therefore maintains a structured memory representation containing profile information, relationships, pets, preferences, projects, programming languages, interests, episodic memories, and conversation summaries. filecite turn0file0 L291-L315

1.2 Project Concept

The project treats the language model as the central reasoning and generation component, while Python functions act as an orchestration layer around it. The orchestration layer decides whether to search the web, whether a message contains a useful long-term fact, whether the user is assigning a nickname to the assistant, and whether a request is asking the system to forget stored information.

1.3 Significance

The significance of the project lies in demonstrating how a general-purpose language model can be turned into a more useful personal application through software engineering. Instead of attempting to retrain a complete AI system for every personal interaction, Zunayra adds external memory and application logic around a locally loaded model.

2. PROBLEM STATEMENT AND MOTIVATION

A conventional chatbot can provide useful answers but may have limited persistent knowledge about an individual user. It may also struggle with two practical requirements: retaining useful information across sessions and obtaining information that changes over time. A purely static local model cannot reliably provide current information such as recent events, prices, software versions, or other time-sensitive facts.

The project therefore addresses the following problem: how can a local personal AI assistant be constructed so that it can converse naturally, retain selected user information, maintain recent conversational context, retrieve current information when required, and expose these capabilities through an accessible user interface?

2.1 Motivation

The motivation is to create an assistant that feels more personal than a stateless question-answering program while keeping the main language-model execution local. The supplied implementation explicitly describes the assistant as a local personal AI assistant and provides configurable model loading, memory files, conversation history, and optional web search. filecite turn0file0 L34-L45

2.2 Key Challenges

- Maintaining useful user information without storing every conversational message as a permanent fact.
- Preventing confusion between facts about the user and the identity of the AI assistant.
- Keeping recent conversational context while preventing history from growing without bound.
- Deciding when external web information is required.
- Handling the case where web search is unavailable because no Tavily API key has been configured.

- Providing an interactive interface without moving the core model logic away from the local application.

3. OBJECTIVES

The primary objective is to develop a functional personal AI assistant that combines local language-model inference with application-level memory and optional web retrieval.

3.1 General Objective

To design and implement a local personal AI assistant capable of natural-language conversation, persistent memory, contextual conversation history, optional current-information retrieval, and a browser-based graphical interface.

3.2 Specific Objectives

- Load a locally stored causal language model and tokenizer using Hugging Face Transformers.
- Generate natural-language responses using configurable generation parameters.
- Maintain structured long-term user memory in a JSON file.
- Extract only significant, long-term facts from user messages.
- Allow the user to assign a nickname to the AI assistant.
- Support explicit requests to forget stored information.
- Store recent conversation turns and summarize older turns when the history limit is reached.
- Determine whether a user query may require fresh web information.
- Retrieve web results through Tavily when web search is enabled.
- Provide a Gradio browser interface resembling a modern chat application.
- Keep the core functionality available through a standalone terminal version.

4. SCOPE OF THE PROJECT

The scope of the project covers the software components represented in the supplied Python implementations. It includes local model loading, text generation, structured memory, conversation history, optional web search, forgetting functionality, and a Gradio-based chat interface.

4.1 Included Scope

Area	Included Functionality
AI Generation	Local causal language model loading and text generation

Memory	Structured JSON memory for selected user facts
Conversation	Recent raw turns plus rolling summaries
Web Retrieval	Optional Tavily search for current information
Identity	Separate AI nickname from user identity
Forget Feature	Targeted removal of remembered facts
UI	Gradio browser-based chat interface
CLI	Standalone terminal-based interaction

4.2 Out of Scope

The source files do not establish a cloud deployment architecture, multi-user authentication system, mobile application, voice interface, image understanding system, or a relational database backend. These may be considered future extensions rather than completed features.

4.3 Intended Users

The system is intended for an individual user who wants a personal conversational assistant running from a local Python environment. The project is also suitable as an academic demonstration of practical AI application integration.

5. EXISTING AND PROPOSED SYSTEM

5.1 Existing Approach

A basic chatbot can be represented as a direct pipeline in which a user message is sent to a language model and the generated response is returned. Such a design has limited persistent state. The model may receive only the current message or a small amount of context, and current web information is not inherently available.

5.2 Proposed System

Zunayra extends the basic chatbot architecture with an orchestration layer. Before generating a final answer, the application can inspect memory, determine whether web search is necessary, build a prompt containing remembered facts and search results, and then invoke the model. After responding, the system can update conversation history and extract long-term memories.

5.3 Comparison

Feature	Basic Chatbot	Zunayra
Local model support	Possible	Implemented
Persistent user memory	Usually absent	JSON-based memory
Conversation history	Limited/optional	Implemented with summarization
Current web information	Not inherent	Optional Tavily search
AI nickname	Not standard	Implemented
Forget capability	Not standard	Implemented
Browser UI	Depends on application	Gradio implementation
CLI interface	Possible	Implemented

6. SYSTEM REQUIREMENTS

6.1 Software Requirements

Component	Requirement
Programming Language	Python
Model Framework	Hugging Face Transformers
Tensor Framework	PyTorch
Acceleration	Accelerate
Web Search Client	tavily-python
Web UI	Gradio
Storage Format	JSON files
Execution Environment	Local Python environment

The standalone implementation identifies Transformers, Torch, Accelerate, and Tavily Python as required libraries. The Gradio version adds Gradio to this set. filecite turn0file0 L18-L20
filecite turn0file1 L11-L12

6.2 Hardware Considerations

The supplied implementation loads the model using automatic data type selection and device mapping. This means actual hardware requirements depend on the size of the local model and the available compute device. The source code itself does not specify a single minimum CPU, GPU,

RAM, or storage configuration; therefore, a fixed hardware specification should not be claimed as a project requirement.

6.3 Configuration Requirements

The model path is configurable through `MODEL_PATH`. The optional web-search component reads `TAVILY_API_KEY` from the environment and enables search only when the key is available.

filecite turn0file0 L31-L45

7. TECHNOLOGY STACK

7.1 Python

Python is used as the primary programming language. It provides the application control flow, file operations, regular-expression processing, JSON serialization, exception handling, and integration with the machine-learning and UI libraries.

7.2 Hugging Face Transformers

The application imports `AutoTokenizer` and `AutoModelForCausalLM`. The tokenizer converts conversation messages into model-compatible input, while the causal language model produces the response tokens. The model is loaded from `MODEL_PATH` using automatic data type selection and device mapping.

filecite turn0file0 L27-L28 filecite turn0file0 L58-L62

7.3 PyTorch

PyTorch provides the runtime used for model generation and device-related operations. The application also checks CUDA availability and clears CUDA cache in selected situations.

7.4 Gradio

Gradio is used in the UI version to create a browser-accessible chat interface. The source defines a `Blocks` application, a `Chatbot` component, a text input, and a `Clear Chat` button.

filecite turn0file1 L529-L541

7.5 Tavily

Tavily is used as an optional external search service. The implementation requests advanced search results and combines returned URLs and content into a text context that can be supplied to the language model.

filecite turn0file0 L71-L84

8. SYSTEM ARCHITECTURE

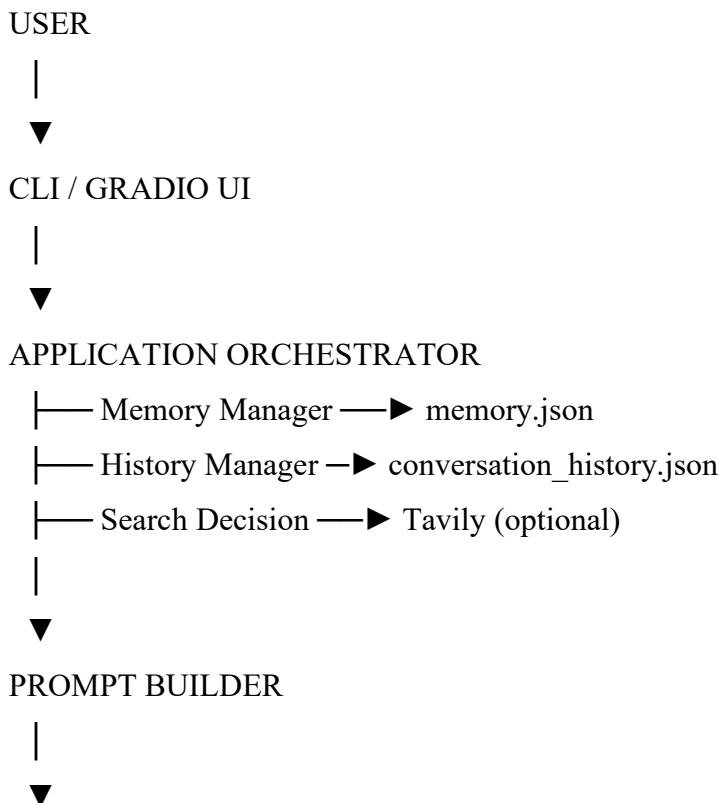
The system follows a layered architecture in which the user interface communicates with an application orchestration layer. The orchestration layer coordinates memory, conversation history, optional web retrieval, prompt construction, and model generation.

8.1 High-Level Architecture

- User Layer: The user interacts through either the terminal application or the Gradio browser interface.
- Application Layer: The main processing pipeline receives the message and decides which supporting operations are required.
- Memory Layer: JSON files store structured profile information, preferences, projects, and other long-term facts.
- History Layer: Conversation turns are stored separately and older turns may be summarized.
- Retrieval Layer: Tavily is invoked when web search is enabled and the request requires fresh information.
- AI Layer: The local tokenizer and causal language model produce the final response.

8.2 Architecture Diagram

Figure 8.1 conceptually represents the main information flow:



LOCAL TOKENIZER + CAUSAL LANGUAGE MODEL



AI RESPONSE

8.3 Runtime Flow

The UI version first displays the user message, then invokes `bot_respond`. The response function performs nickname detection, optional web search, prompt construction, model generation, and memory extraction. This sequencing is explicitly implemented through Gradio's `submit` callback followed by `bot_respond`. filecite turn0file1 L458-L526

9. SYSTEM DESIGN

9.1 Functional Requirements

ID	Requirement
FR-01	The system shall load a configured local language model.
FR-02	The system shall accept natural-language user messages.
FR-03	The system shall generate an AI response.
FR-04	The system shall store selected long-term user facts.
FR-05	The system shall maintain recent conversation history.
FR-06	The system shall summarize older history when the configured limit is exceeded.
FR-07	The system shall optionally retrieve current information from the web.
FR-08	The system shall distinguish the user's identity from the AI's nickname.
FR-09	The system shall support forgetting a selected remembered fact.
FR-10	The browser version shall provide a chat interface and clear-chat control.

9.2 Non-Functional Requirements

- Usability: The Gradio interface should provide a simple conversational interaction model.
- Maintainability: Functions are separated by responsibilities such as chat, memory, search, and history.
- Reliability: The code includes exception handling for model/search and JSON-related operations.
- Privacy-oriented design: Long-term memory is stored in local JSON files rather than requiring a dedicated remote database.
- Configurability: Model path and web-search availability are controlled through configuration values and environment variables.

9.3 Use Case Summary

Actor	Use Case	Description
User	Chat	Send a natural-language message and receive an AI response.
User	Provide Personal Fact	Tell the assistant a fact that may be useful in future conversations.
User	Name Assistant	Assign a nickname to the AI.
User	Forget Fact	Request removal of a remembered fact.
User	Request Current Information	Ask a question that may trigger web search.
User	Clear Chat	Clear the visible Gradio chat history.

10. MODULE DESCRIPTION

10.1 Model Loading Module

The model-loading section initializes the tokenizer and causal language model from MODEL_PATH. It verifies that the path exists and exits with an error message if it does not. This module establishes the model used throughout the session.

10.2 Chat Generation Module

The chat function constructs a system instruction, appends optional conversation history, adds the current user message, applies the tokenizer's chat template, and generates up to 200 new tokens. Sampling is enabled with temperature 0.3 and top-p 0.9. filecite turn0file0 L92-L165

10.3 Web Search Module

The web_search function first checks whether WEB_SEARCH_ENABLED is true. When enabled, it creates a TavilyClient and requests up to five advanced search results. Each result contributes a source URL and content to the context passed back to the application. filecite turn0file0 L71-L84

10.4 Memory Decision Module

The should_extract_memory function asks the language model to classify whether a message contains information important enough for long-term memory. This separates trivial conversation from potentially useful user facts.

10.5 AI Nickname Module

The extract_ai_nickname function is deliberately designed to distinguish statements about the user from statements that assign a name to the AI. For example, a statement describing the user's own age is explicitly excluded from nickname extraction. filecite turn0file0 L229-L283

10.6 Memory Storage Module

The memory manager initializes a default structured memory object and loads existing memory.json data when available. The system can save the updated memory as formatted JSON. filecite turn0file0 L287-L347

10.7 Forget Module

The forget mechanism identifies whether the user is requesting removal of a stored fact. It can target selected fields such as a favorite color or name and can clear an entire list in supported cases. filecite turn0file0 L597-L647

10.8 History Module

Conversation history is stored in conversation_history.json. The implementation keeps the most recent six user-plus-AI exchanges verbatim and summarizes older content into

conversation_summaries when the history grows beyond that limit. filecite turn0file0 L472-L520

10.9 Gradio UI Module

The UI module creates a Gradio Blocks application with a title, optional web-search status message, a chatbot, text input, and Clear Chat button. filecite turn0file1 L529-L542

11. IMPLEMENTATION DETAILS

11.1 Model Initialization

At startup, the system prints a loading message, checks MODEL_PATH, loads the tokenizer, and loads the causal language model. The UI version performs this loading once before the interface starts, preventing model initialization on every user message.

11.2 Prompt Construction

The build_prompt function converts structured memory into readable text sections. It also includes fresh web-search results when they exist. The resulting prompt contains an AI identity line, known user facts, optional search context, the current message, and response instructions. filecite turn0file0 L523-L568

11.3 Conversation Processing

For a normal request, the application loads memory, determines whether search should occur, builds a prompt, generates the response, appends the user and assistant messages to history, and then attempts memory extraction. The standalone main loop implements this pipeline directly. filecite turn0file0 L651-L738

11.4 Web Search Decision

The decision mechanism uses both explicit keywords and an LLM-based classifier. Explicit keywords include search, online, check, latest, current, and google. The classifier is instructed to return SEARCH for time-sensitive information and NO_SEARCH for personal questions, general knowledge, logic/math, or greetings. The implementation also checks NO_SEARCH before SEARCH to avoid substring ambiguity. filecite turn0file0 L199-L225

11.5 Memory Extraction Format

The memory extractor requests a JSON array containing category, field, and value. It supports categories including profile, relationships, pets, preferences, projects, programming languages, interests, episodic memories, and conversation summaries. The returned JSON is validated before being merged into the stored memory. filecite turn0file0 L350-L415

11.6 Memory Merge Strategy

The merge function handles both dictionary fields and list fields. New list values are appended only when they are not already present. Single-value fields can be updated when a newer value differs. This design reduces unnecessary duplication in persistent memory. filecite turn0file0 L420-L468

12. DATA STORAGE AND MEMORY DESIGN

The system uses lightweight JSON files rather than a database server. This is appropriate for a single-user local assistant because the data structures are relatively simple and can be loaded and saved directly by Python.

12.1 Main Storage Files

File	Purpose
memory.json	Persistent structured user and assistant memory
conversation_history.json	Stored conversation messages
last_fact.json	Reference to the most recently extracted fact for convenient forget handling

12.2 Memory Schema

Category	Examples from Implementation
profile	name, preferred_name, birthday, country, hobbies, skills, siblings
relationships	friends, family
pets	pet list
preferences	favorite_food, favorite_player, favorite_color, favorite_game, favorite_movie, favorite_book
projects	projects

programming_languages	programming languages
interests	interests
episodic_memories	episodic memories
conversation_summaries	summaries of older conversations
ai_profile	AI nickname

12.3 Memory Lifecycle

1. User sends a message.
2. The system decides whether it may contain long-term information.
3. If yes, the memory extractor produces structured JSON.
4. The JSON result is validated.
5. The extracted facts are merged into the current memory object.
6. The updated memory is saved to memory.json.
7. The latest fact is optionally recorded in last_fact.json.

12.4 Conversation Lifecycle

The conversation history is separate from long-term memory. Each successful response appends both the user message and assistant response. When the history exceeds the configured raw-history capacity, the older portion is summarized and the recent messages are retained. This distinction prevents the assistant from treating every previous sentence as a permanent personal fact.

13. USER INTERFACE DESIGN

13.1 Standalone Interface

The standalone application provides a terminal interaction model. After model loading, the user receives a prompt and can type messages until entering the exit command. The application also reports when web search is unavailable. filecite turn0file0 L656-L670

13.2 Gradio Interface

The Gradio version is intended to provide a more accessible user experience. It creates a browser-based chat page with a chatbot display, a message input, and a clear button. The interface launches in the browser automatically using inbrowser=True. filecite turn0file1 L529-L542

13.3 Immediate User Message Display

An important UI behavior is implemented in `user_submit`. When the user presses Enter, the message is added to the visible chat immediately and the text box is cleared. Model processing is then performed by `bot_respond` through a chained callback. This design makes the interface feel more responsive because the user's message does not wait for model generation before appearing.

13.4 UI Flow

Type Message → Submit



Display User Message Immediately



Check Nickname / Search / Memory



Generate Model Response



Display AI Response



Update Persistent Memory

13.5 UI Limitations

The supplied Gradio code provides a functional chat interface but does not show advanced settings panels, user authentication, multiple conversations, file upload, voice interaction, or a dedicated memory-management screen. These are potential future improvements rather than existing features.

14. ALGORITHMS AND PROCESSING FLOW

14.1 Main Response Algorithm

8. Receive the user's message.
9. Check whether the message assigns a nickname to the AI.
10. Check whether the message requests a memory item to be forgotten.
11. Load the current memory context.
12. Determine whether web search is required.
13. If enabled and required, perform a Tavily search.

14. Build a prompt containing relevant memory and search context.
15. Generate the response using the local causal language model.
16. Display the response.
17. Append the exchange to conversation history.
18. Summarize older history if necessary.
19. Determine whether the message contains a long-term fact.
20. Extract, validate, merge, and save new memory.

14.2 Pseudocode

START

Load memory

Load conversation history

WHILE user has not typed exit:

 RECEIVE user_message

 IF user_message assigns AI nickname:

 save nickname

 display confirmation

 CONTINUE

 IF user_message asks to forget:

 identify target

 remove target if valid

 display result

 CONTINUE

memory_context = load memory

IF web search enabled AND query requires fresh information:

 search_results = Tavily.search(user_message)

ELSE:

 search_results = none

prompt = build_prompt(user_message, memory_context, search_results)

response = local_language_model(prompt)

display response

save user_message + response

IF history exceeds limit:

summarize old messages
save summary
retain recent messages

IF user_message contains important long-term fact:
facts = extract_memory(user_message)
validate facts
merge facts into memory
save memory

END

14.3 Forget Algorithm

The forget process first attempts to use the most recently saved fact. If that is not sufficient, the system asks the model to identify a target category and field from supported memory fields. The selected field is then cleared and the updated memory is saved.

14.4 Search Decision Algorithm

The search decision combines deterministic keyword detection with an LLM classification step. The deterministic layer is useful when the user explicitly requests an online or current lookup. The classifier provides a broader semantic decision for time-sensitive queries.

15. TESTING AND EVALUATION

Testing for this project should focus on functional behavior rather than claiming performance metrics that are not provided by the source files. The following test cases are derived from the implemented features.

Test ID	Test Case	Expected Result
TC-01	Start with valid MODEL_PATH	Model and tokenizer load successfully.
TC-02	Start with invalid MODEL_PATH	Application reports missing model path and exits.
TC-03	Send normal message	AI response is generated.
TC-04	Assign AI nickname	Nickname is stored in ai_profile.
TC-05	State user fact	Fact may be extracted and saved if classified as important.

TC-06	Ask to forget recent fact	Relevant memory field is cleared.
TC-07	Ask current-information question with API key	Web search can be performed.
TC-08	Run without Tavily API key	Chat remains available while web search is disabled.
TC-09	Send many messages	Older history is summarized when configured threshold is exceeded.
TC-10	Open Gradio version	Browser chat interface is launched.
TC-11	Press Clear Chat	Visible Gradio chat history is cleared.
TC-12	Ask about user vs. AI identity	Grounding instructions aim to keep the two identities separate.

15.1 Reliability Considerations

The implementation includes exception handling for malformed JSON, search failures, memory extraction failures, and selected GPU out-of-memory conditions. These mechanisms do not guarantee complete reliability, but they reduce the likelihood that one failed auxiliary operation will terminate the entire conversational workflow.

15.2 Evaluation Approach

A complete academic evaluation could additionally measure response latency, memory extraction precision, search-trigger accuracy, model response quality, and resource consumption across representative hardware. Such quantitative measurements are not present in the supplied source and therefore are not fabricated in this report.

16. SECURITY, PRIVACY, AND RELIABILITY CONSIDERATIONS

16.1 Local Data Storage

The long-term memory and conversation history are stored in local JSON files. This reduces the need for a remote database for the core memory mechanism. However, local files still require normal operating-system access controls and responsible handling because they may contain personal information.

16.2 API Key Handling

The Tavily API key is read from an environment variable rather than being hard-coded in the main source file. This is a preferable configuration practice because it reduces the chance of accidentally embedding a credential directly in source code. `filecite turn0file0 L38-L40`

16.3 Model Grounding

The system includes explicit instructions that the model should only state user facts that are actually present in conversation history or known memory. The implementation also tells the model not to fabricate explanations about previous behavior when it is uncertain. `filecite turn0file0 L123-L139`

16.4 Limitations of the Approach

Prompt-level grounding is not equivalent to formal verification. A language model can still produce incorrect output. Similarly, the memory extractor depends on model-generated structured JSON, so validation and cautious handling are necessary. The project includes basic validation, but a production-grade system would require stronger schema validation, access control, encryption, and audit mechanisms.

17. ADVANTAGES AND LIMITATIONS

17.1 Advantages

- Local model execution can reduce dependence on a hosted conversational model for the core response generation.
- Persistent memory allows the assistant to retain selected information between executions.
- Conversation summaries prevent the raw conversation history from growing without bound.
- Optional web search provides a mechanism for current information when enabled.
- The AI nickname mechanism creates a clear distinction between assistant identity and user identity.
- The forget feature gives the user a way to remove selected remembered information.
- The Gradio UI provides a more approachable experience than the terminal interface.
- The modular Python structure separates model, memory, search, history, and UI responsibilities.

17.2 Limitations

- Actual response quality depends strongly on the locally selected language model.

- Model resource requirements vary with model size and are not fixed by the application code.
- Web search is unavailable unless a Tavily API key is configured.
- Memory extraction depends on model classification and structured output.
- JSON files are suitable for a single-user prototype but are not a substitute for a robust multi-user database.
- The supplied UI does not implement authentication or separate user accounts.
- The source does not provide quantitative benchmark results for latency, accuracy, or memory quality.
- The system does not include a dedicated graphical memory-management interface.

18. FUTURE ENHANCEMENT

The existing implementation provides a strong prototype foundation. Several improvements could extend it toward a more complete personal AI product.

- Add a dedicated memory dashboard where users can view, edit, and delete stored facts.
- Introduce stronger JSON schema validation or migrate persistent data to SQLite for better integrity.
- Add encrypted local storage for sensitive memory and history.
- Support multiple user profiles with authentication.
- Add configurable model parameters through the UI.
- Add streaming token output so responses appear progressively.
- Add source citation cards for web-search results.
- Add voice input and text-to-speech output.
- Add file/document question answering using a retrieval pipeline.
- Add automated evaluation datasets for memory extraction and search-trigger accuracy.
- Add logging and diagnostic tools for model latency and resource usage.
- Package the application for easier installation and distribution.

18.1 Suggested Advanced Architecture

A future version could separate the system into an application server, model service, memory service, retrieval service, and UI layer. A vector database could be added for semantic retrieval while structured profile facts remain in a relational or document store. Such an architecture would allow more scalable memory retrieval without changing the basic conceptual pipeline demonstrated by the current project.

19. CONCLUSION

Zunayra demonstrates how a local causal language model can be transformed into a practical personal AI assistant through software orchestration. Rather than relying on the language model alone, the project surrounds it with persistent memory, conversation history, optional web retrieval, identity handling, forgetting functionality, and an interactive Gradio interface.

The implementation successfully separates several responsibilities into dedicated functions. The model handles natural-language generation; the memory subsystem manages persistent user facts; the history subsystem manages conversational context; the search subsystem supplies optional current information; and the Gradio layer provides browser interaction. This modular approach makes the project suitable for academic demonstration and further development.

An important characteristic of the project is that it does not treat every conversation as permanent memory. Instead, it asks the model to identify information that is significant enough for long-term storage and then stores the extracted result in a structured representation. The system also explicitly distinguishes user identity from the AI's own nickname, which addresses a common source of conversational confusion.

Overall, the project provides a practical foundation for a personal AI assistant and demonstrates concepts from artificial intelligence, natural language processing, software architecture, persistent data management, and human-computer interaction. Its main opportunities for future work are stronger data security, more robust memory management, quantitative evaluation, richer interfaces, and more scalable retrieval.

20. REFERENCES

- Project Source File: Zunayra — Personal AI Assistant (standalone local version). Supplied project source code. `filecite turn0file0 L2-L20`
- Project Source File: Zunayra — Chat UI version (Gradio, ChatGPT/Gemini-style). Supplied project source code. `filecite turn0file1 L2-L12`
- Hugging Face Transformers library — used through `AutoTokenizer` and `AutoModelForCausalLM` in the supplied implementation.
- PyTorch — used for model execution and device management in the supplied implementation.
- Gradio — used to construct the browser-based chat interface in the supplied implementation.
- Tavily Python client — used for optional web-search functionality in the supplied implementation.

21. Accuracy

Full MMLU Evaluation Complete.

Total questions evaluated across all subjects: 28084

Overall Accuracy on MMLU: 69.79%

Accuracy per subject:

- abstract_algebra: 57.00%
- all: 69.83%
- anatomy: 66.67%
- astronomy: 82.24%
- business_ethics: 77.00%
- clinical_knowledge: 76.60%
- college_biology: 85.42%
- college_chemistry: 54.00%
- college_computer_science: 72.00%
- college_mathematics: 47.00%
- college_medicine: 75.72%
- college_physics: 55.88%
- computer_security: 77.00%
- conceptual_physics: 79.15%
- high_school_biology: 90.65%
- high_school_chemistry: 74.88%
- high_school_computer_science: 86.00%
- high_school_european_history: 78.79%
- high_school_geography: 84.34%
- moral_disputes: 70.52%
- moral_scenarios: 37.77%

- nutrition: 76.47%
- philosophy: 73.63%
- prehistory: 75.93%
- professional_accounting: 59.22%
- professional_law: 48.17%
- professional_medicine: 76.10%
- professional_psychology: 71.24%

APPENDIX A: IMPORTANT CODE STRUCTURES

A.1 Configuration

```
MODEL_PATH = "."  
TAVILY_API_KEY = os.environ.get("TAVILY_API_KEY")  
WEB_SEARCH_ENABLED = bool(TAVILY_API_KEY)
```

```
MEMORY_FILE = "memory.json"  
HISTORY_FILE = "conversation_history.json"  
LAST_FACT_FILE = "last_fact.json"  
MAX_RAW_HISTORY_TURNS = 6
```

A.2 Model Loading

```
tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH)  
model = AutoModelForCausalLM.from_pretrained(  
    MODEL_PATH,  
    torch_dtype="auto",  
    device_map="auto",  
)
```

A.3 Gradio Interface Structure

```
with gr.Blocks(title="Zunayra") as demo:  
    gr.Markdown("# Zunayra — Personal AI Assistant")  
    chatbot = gr.Chatbot(height=500)  
    msg = gr.Textbox(placeholder="Type your message here...")  
    clear = gr.Button("Clear Chat")  
  
    msg.submit(  
        lambda msg: chatbot.submit(msg, clear, lambda: None),  
        msg, chatbot, clear, lambda: None
```

```

        user_submit,
        [msg, chatbot],
        [msg, chatbot]
    ).then(
        bot_respond,
        chatbot,
        chatbot
    )

clear.click(lambda: [], None, chatbot)

```

A.4 Source-Based Implementation Summary

The standalone source defines the main loop, model generation, memory extraction, memory merging, history summarization, search decision, web search, and forget functionality. `filecite turn0file0 L651-L738`

The Gradio source reuses the core architecture while adding immediate user-message rendering and a browser interface. `filecite turn0file1 L458-L526`

APPENDIX B: PROJECT FILE ORGANIZATION

Component	Role
<code>zunayra_app.py</code>	Standalone local/terminal version represented by the supplied source.
<code>zunayra_ui.py</code>	Gradio browser-interface version represented by the supplied source.
<code>memory.json</code>	Runtime persistent memory file.
<code>conversation_history.json</code>	Runtime conversation-history file.
<code>last_fact.json</code>	Runtime reference to the latest extracted fact.
Local model directory	Contains tokenizer/model files required by <code>MODEL_PATH</code> .

The exact filenames for runtime files are defined in the source code. The model directory is configurable through `MODEL_PATH` rather than fixed to one absolute location.